

Shortest Paths

COMP4128 Programming Challenges

School of Computer Science and Engineering
UNSW Sydney

Term 2, 2026

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit
Graphs

1 Single Source Shortest Paths

- Dijkstra's Algorithm
- Bellman-Ford Algorithm

2 All Pairs Shortest Paths

3 Implicit Graphs

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit
Graphs

- Given a weighted directed graph G with two specific vertices s and t , we want to find the shortest path that goes between s and t on the graph.
- Note that the unweighted case is solved by BFS.
- Generally, algorithms which solve the shortest path problem also solve the single source shortest path problem, which computes shortest paths from a single source vertex to every other vertex.
- You can represent all the shortest paths from the same source as a tree.

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit
Graphs

- It's very important to distinguish between graphs where all edges are positive and graphs with negative weight edges! Why?
- Imagine a graph with a cycle whose total weight is negative.
- Even if there are no negative cycles, this may still cause problems depending on your algorithm choice!
- If the graph is acyclic, negative weight edges generally don't cause problems, but care should be taken regardless.

Shortest Paths

Single Source Shortest Paths

Dijkstra's Algorithm

Bellman-Ford Algorithm

All Pairs Shortest Paths

Implicit Graphs

- Most single source shortest paths algorithms rely on the basic idea of building shortest paths iteratively. At any point, we keep what we think is the shortest path to each vertex and we update this by “relaxing” edges.
- $\text{Relax}(u, v)$: if the currently found shortest path from our source s to a vertex v could be improved by using the edge (u, v) , update it.
- For graphs with non-negative weights, we can get away with only relaxing vertices for which we know the optimal distance. But with negative weights, this becomes trickier.

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit
Graphs

- If we keep track for each v of its most recently relaxed incoming edge, we can find the actual path from the source to v . How?
- For each v , we know the vertex we would've come from if we followed the shortest path from the source.
- We can work backwards from v to the source to find the shortest path *from* the source *to* v .

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit
Graphs

- If we keep relaxing our edges until we can't anymore, then we will have a shortest path.
- How do we choose which edges to relax?

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- For now, suppose there are no negative edges.
- Visit each vertex u in turn, starting from the source s .
Whenever we visit the vertex u , we relax all of the edges coming out of u .
- How do we decide the order in which to visit each vertex?

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- We can do something similar to breadth-first search.
- The next vertex we process is always the unprocessed vertex with the smallest distance from the source.
- This ensures that we only need to process each vertex once. Why?
 - Claim: by the time we process a vertex, we have definitely found the shortest path to it.
 - Proof: by induction, left as an exercise.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- To decide which vertex we want to visit next, we can either just loop over all of them, or use a priority queue keyed on each vertex's current shortest known distance from the source.
- Since we know that we have a complete shortest path to every vertex by the time we visit it in Dijkstra's algorithm, we know we only visit every vertex once.
- **Complexity** Dijkstra's Algorithm is $O(E \log V)$ using a binary heap as a priority queue, or $O(V^2)$ with a loop (i.e. linear search for the min)

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- The above only holds for graphs without negative edges!
- With negative edges, we may need to visit each vertex more than once, and it turns out this makes the runtime exponential in the worst case (and it's even worse with negative cycles).
- In short: don't use Dijkstra's if there's any negative edges! (But most graphs you see won't have them).

Shortest Paths

Single Source Shortest Paths

Dijkstra's Algorithm

Bellman-Ford Algorithm

All Pairs Shortest Paths

Implicit Graphs

```
#include <algorithm> // fill
#include <queue>
#include <utility>    // pair
using namespace std;

typedef pair<int, int> edge; // (distance, vertex)

const int INF = 1000*1000*1000+7; // more than any real distance
const int N = 100100;
vector<edge> edges[N]; // assume prefilled adjlist
int dist[N]; // dist[i] = INF denotes unseen

void dijkstra (int s) {
    fill(dist, dist+N, INF);
    priority_queue<edge, vector<edge>, greater<edge>> pq;
    pq.emplace(0, s); // distance to s itself is zero
    while (!pq.empty()) {
        // choose (d, v) so that d is minimal
        // i.e. the closest unvisited vertex
        edge cur = pq.top(); // since C++17: auto [d, v] = pq.top();
        int v = cur.second, d = cur.first;
        pq.pop();
        if (dist[v] == INF) { // if not seen
            dist[v] = d;
            for (edge nxt : edges[v]) { // relax all edges from v
                int u = nxt.second, w = nxt.first;
                if (dist[u] == INF) { pq.emplace(d + w, u); }
            }
        }
    }
}
```

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- How do we handle negative edges in a more efficient way?
- How do we handle negative cycles?
- **Key Observation:** If a graph has no negative cycle then all shortest paths from u have length $\leq |V| - 1$.
Conversely, a negative cycle implies there is a shortest path of length $|V|$ better than any path of length $|V| - 1$.
- So we should instead build up shortest paths by number of edges, not just from u outwards.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Bellman-Ford involves trying to relax every edge of the graph (a *global relaxation*) $|V| - 1$ times and update our tentative shortest paths each time.
- Because every shortest path has at most $|V| - 1$ edges, if after all of these global relaxations, relaxations can still be made, then there exists a negative cycle.
- We can now proceed using dynamic programming.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Subproblem: $\text{dist}(v, k)$, the shortest $s \rightarrow v$ path using up to k edges
- Recurrence:

$$\text{dist}(v, k) = \min \left[\text{dist}(v, k-1), \min_{u: e=(u,v) \in E} \left(\text{dist}(u, k-1) + w_e \right) \right]$$

- Base cases: $\text{dist}(v, 0) = 0$ for $v = s$, or ∞ otherwise
- Time complexity is $O(VE)$ (since we do $O(E)$ work total for each k)
- Shortest path lengths are $\text{dist}(\cdot, |V| - 1)$

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- There are $O(V^2)$ subproblems, so do we need $O(V^2)$ space?
- No! $\text{dist}(\cdot, k)$ only depends on $\text{dist}(\cdot, k - 1)$, so we can keep only two rows and alternate between them.
- Better yet, we can keep just one row, and overwrite it, replacing $\text{dist}(u, k - 1)$ with $\text{dist}(u, k)$ in the recurrence.
 - If there are no negative cycles, the intermediate results might change but the final answer will be the same, since using more than $|V| - 1$ edges is never necessary.
 - If there is a negative cycle, we will still detect it.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- If we have some way of knowing that the last global relaxation did not affect the tentative shortest path to some vertex v , we know that we don't need to consider edges coming out of v in our next global relaxation.
- This heuristic doesn't change the overall time complexity of the algorithm, but makes it run very fast in practice on random graphs.
- Sometimes called Shortest Path Faster Algorithm (SPFA)

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

```
struct edge {
    int u, v, w; // u -> v of weight w
    edge(int _u, int _v, int _w) : u(_u), v(_v), w(_w) {}
};

vector<int> dist(n);
vector<edge> edges;

// global relaxation: try to relax every edge in the graph
// returns whether any distance was updated
bool relax() {
    bool relaxed = false;
    for (edge e : edges) {
        // we don't want to relax an edge from an unreachable vertex
        if (dist[e.u] != INF && dist[e.v] > dist[e.u] + e.w) {
            relaxed = true;
            dist[e.v] = dist[e.u] + e.w;
        }
    }
    return relaxed;
}
```

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

```
// Puts distances from source vertex 0 in dist  
// Returns true if there is a negative cycle, false otherwise.  
// NOTE: You can't trust the dist array if this function returns True.  
bool bellman_ford() {  
    fill(dist.begin(), dist.end(), INF);  
    dist[0] = 0;  
    // V-1 global relaxations  
    // if no updates are made in an entire round, we can early exit  
    // SPFA optimises this further  
    for (int i = 0; i < n - 1; i++) {  
        if (!relax()) { break; }  
    }  
    // there is a negative cycle iff any edge can be relaxed further  
    // therefore try a Vth global relaxation  
    // true if any changes made, false otherwise  
    // can micro-optimize by early exiting when the first change is made  
    return relax();  
}
```

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- If there is a negative cycle, you can't trust the distances computed.
- Call a vertex v 'ruined' if its shortest distance from u is actually $-\infty$.
- For every negative cycle, in every relaxation round at least one of its vertices will be updated.
- Hence, to find all ruined vertices, DFS out of all vertices who were relaxed in the V -th round.
- To find a specific negative cycle, backtrack from any 'ruined' vertex.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

1 Single Source Shortest Paths

- Dijkstra's Algorithm
- Bellman-Ford Algorithm

2 All Pairs Shortest Paths

3 Implicit Graphs

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit
Graphs

- The all pairs shortest path problem involves finding the shortest path between every pair of vertices in the graph.
- Surprisingly, this can be found in $O(V^3)$ time and $O(V^2)$ memory, by dynamic programming.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Let $\text{dist}(u, v, i)$ be the length of the shortest path between u and v using only the first i vertices (i.e. the vertices with the i smallest labels) as intermediate vertices.
- The key is to build this up for increasing values of i .
- **Base Case:** Then $\text{dist}(u, u, 0) = 0$ for all vertices u , and $\text{dist}(u, v, 0) = w_e$ if there is an edge e from u to v , and infinity otherwise.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Say we have already calculated $\text{dist}(u, v, i - 1)$ for all pairs u, v and some i . Then

$$\text{dist}(u, v, i) = \min \left[\text{dist}(u, v, i - 1), \right. \\ \left. \text{dist}(u, i, i - 1) + \text{dist}(i, v, i - 1) \right].$$

- The solution we already had, $\text{dist}(u, v, i - 1)$, definitely doesn't use i as an intermediate vertex.
- If i is the only new intermediate vertex we can use, the only new path that could be better is the shortest path $u \rightarrow i$ concatenated with the shortest path $i \rightarrow v$.

Shortest Paths

Single Source
Shortest PathsDijkstra's
AlgorithmBellman-Ford
AlgorithmAll Pairs
Shortest PathsImplicit
Graphs

$$\text{dist}(u, v, i) = \min \left[\text{dist}(u, v, i-1), \text{dist}(u, i, i-1) + \text{dist}(i, v, i-1) \right]$$

- Thus, $\text{dist}(u, v, n)$ will give the length of the shortest path from u to v .
- Noting that to calculate the table for the next i , we only need the previous table, we see that we can simply overwrite the previous table at each iteration, so we only need $O(V^2)$ space.
- But what if $\text{dist}(u, i, i-1)$ or $\text{dist}(i, v, i-1)$ is overwritten in the table before we get to use it?
- Allowing the use of i as an intermediate vertex on a path to or from i is not going to improve the path, unless we have a negative-weight cycle.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

```
// the distance between everything is infinity
for (int u = 0; u < n; ++u)
    for (int v = 0; v < n; ++v) { dist[u][v] = INF; }

// update the distances for every directed edge
// each edge u -> v with weight w
for (edge e : edges) { dist[e.u][e.v] = e.w; }

// every vertex can reach itself
for (int u = 0; u < n; ++u) { dist[u][u] = 0; }

for (int i = 0; i < n; i++) { // zero-based
    for (int u = 0; u < n; u++) {
        for (int v = 0; v < n; v++) {
            // dist[u][v] is the length of the shortest path from u to v using only 0
            to i as intermediate vertices
            // now that we're allowed to also use i, the only new path that could be
            shorter is u -> i -> v
            dist[u][v] = min(dist[u][v], dist[u][i] + dist[i][v]);
        }
    }
}
```

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit
Graphs

- If there is a negative-weight cycle, our invariant is instead $\text{dist}(u, v, i) \leq$ shortest simple path from $u \rightarrow v$ only using the first i vertices as intermediaries.
- Hence $\text{dist}(u, u, n)$ will be negative for vertices in negative cycles. Also you can't trust the calculated distances, same as Bellman-Ford.
- Note that if there *are* negative-weight edges, but *no negative cycles*, Floyd-Warshall will correctly find all distances.
- This distinction is only necessary for directed graphs; a negative weight edge in an undirected graph immediately gives rise to a 2-cycle.

Floyd-Warshall Algorithm: Reconstructing Paths 28

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- How can we find the actual shortest path?
- As well as keeping track of a `dist` table, any time the improved path (via i) is used, note that the next thing on the path from u to v is going to be the next thing on the path from u to i , which we should already know because we were keeping track of it!
- When initialising the table with the edges in the graph, don't forget to set v as next on the path from u to v for each edge $u \rightarrow v$.
- Implementing this functionality is left as an exercise.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- 1 Single Source Shortest Paths
 - Dijkstra's Algorithm
 - Bellman-Ford Algorithm
- 2 All Pairs Shortest Paths
- 3 Implicit Graphs

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Although some graph interpretations are obvious (e.g. cities and highways), it's often the case that the graph you must run your algorithm on is non-obvious.
- Often this doesn't admit a clean implementation using something like an explicit adjacency list.
- In many cases like this, it may be a better idea to compute the adjacencies on the fly.
- Also, sometimes modifying the graph you consider is helpful!

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Problem Statement** You have found a strange device that has a red button, a blue button, and a display showing a single integer, initially n . Pressing the red button multiplies the number by two; pressing the blue button subtracts one from the number. If the number stops being positive, the device breaks. You want the display to show the number m . What is the minimum number of button presses to make this happen?
- **Input** Two space-separated integers n and m ($1 \leq n, m \leq 10^7$).
- **Output** A single number, the smallest number of button presses required to get from n to m .

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- In this example, we should think of our button presses as transitions.
- Hence our graph has numbers as its vertices and edges representing which numbers can reach each other through button presses.
- The graph is unweighted, so we just need to do a simple BFS to find the answer.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- However, there are so many positive integers!
 - Perhaps there might be more possible vertices than we can store in contiguous memory.
 - What can the actual solution look like? What intermediate values can appear on the display?
- If $n \geq m$, doubling doesn't help, so we just decrement $n - m$ times.
- However the $n < m$ case is more complicated!
 - We might need to use both buttons.
 - However, we should never decrement twice in a row after the first doubling, since $n \rightarrow 2n \rightarrow 2n - 1 \rightarrow 2n - 2$ takes three moves to achieve the same as $n \rightarrow n - 1 \rightarrow 2n - 2$.
 - Therefore we never display a value larger than $m + 1$.
 - Note however that the answer can still be linear in n ; consider for example $15 \rightarrow 14 \rightarrow \dots \rightarrow 8 \rightarrow 16$.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

Three ways to implement this:

- 1 Construct an explicit distance array (since there are only 10^7 vertices) and explicit adjacency list (since there are only 2×10^7 edges).
- 2 Construct an explicit distance array but construct the outgoing edges from a vertex at the time it is popped from the BFS queue (since they are easy to calculate).
- 3 Construct a distance map (so only the actually explored vertices are added) and construct the outgoing edges from a vertex at the time it is popped from the BFS queue.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- The last of these approaches is too slow (what is the counterexample?).
- The first two approaches are both fine; we've chosen the second for instructional value.
- If there were many more theoretically possible vertices but fewer of them were actually encountered, the first two approaches could run out of memory and the third approach could be the only option.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

● Implementation

```
#include <algorithm>
#include <iostream>
#include <queue>
using namespace std;

const int MAXVAL=20*1000*1000+5;
const int INF = 1000*1000*1000+7;
int n, m, v[MAXVAL];
queue<int> q;

int main () {
    cin >> n >> m;
    fill(v, v + MAXVAL, INF);
    q.push(n);
    v[n] = 0;
    while (!q.empty()) {
        int i = q.front(); q.pop();
        if (i-1 > 0 && v[i] + 1 < v[i-1]) {
            v[i-1] = v[i] + 1;
            q.push(i - 1);
        }
        if (i*2 < MAXVAL && v[i] + 1 < v[i*2]) {
            v[i*2] = v[i] + 1;
            q.push(i * 2);
        }
    }
    cout << v[m] << '\n';
}
```

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Problem Statement** You are a rock climber trying to climb a wall. On this wall, there are n rock climbing holds for you to use. Whenever you are on the wall, you must be holding on to exactly three holds, each of which can be at most D distance from the other two. To move on the wall, you can only disengage from one of the holds and move it to another hold that is within D distance of the two holds that you are still holding onto. You can move from hold to hold at a rate of 1m/s. How can you reach the highest hold in the shortest amount of time, starting from some position that includes the bottom hold?
- **Input** A set of up to n ($1 \leq n \leq 50$) points on a 2D plane, and some integer D ($1 \leq D \leq 1000$). Each point's coordinates will have absolute value less than 1,000,000.
- **Output** A single number, the least amount of time needed to move from the bottom to the top.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- If there was no restriction that required you to always be using three holds, then this would just be a standard shortest path problem that is solvable using Dijkstra's algorithm.
- We would just need to take the points as the vertices and the distance between points as the edge weights.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- However, we need to account for the fact that we must be using three holds clustered together at any time.
- But there is a natural interpretation of the hold restriction in terms of a graph: when we move from some position that uses holds $\{a, b, c\}$ to some position where we use holds $\{a, b, d\}$, we can say that we are moving from some vertex labelled $\{a, b, c\}$ to some vertex labelled $\{a, b, d\}$.
- It can be determined whether or not such a move is allowed, i.e. if there is an edge between these vertices, in constant time.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Now, we have a graph where we have $O(n^3)$ vertices and $O(n^4)$ edges.
- Running Dijkstra's algorithm on this graph directly will give us the answer we want, by definition.
- So we can solve this problem in $O(E \log V) = O(n^4 \log n^3) = O(n^4 \log n)$ time.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

● Implementation

```
struct state {
    int id[3];
    double dist;
    state(int a, int b, int c, double d): id{a,b,c}, dist(_d) {}
};

bool operator< (const state &a, const state &b) {
    // reversing the comparison makes pq a min heap by default
    return a.dist > b.dist;
}

double shortest_climb (void) {
    priority_queue<state> pq;
    // push all states containing hold 1, at distance 0
    for (int i = 2; i <= n; i++) {
        for (int j = i+1; j <= n; j++) {
            if (dist(1,i) <= D && dist(1,j) <= D && dist(i,j) <= D)
                seen[1][i][j] = 0;
            pq.push(state(1,i,j,0));
        }
    }

    while (!pq.empty()) {
        state cur = pq.top();
        pq.pop();
        // check if done
        if (cur.id[2] == n) { return cur.dist; }
```

• Implementation (continued)

```

// try disengaging our jth hold
for (int j = 0; j < 3; j++) {
    // and moving to hold number i
    for (int i = 1; i <= n; i++) {
        for (int k = 0; k < 3; k++) {
            // can't reuse existing holds
            if (i == cur.id[k] ||
                // or be too far from retained holds
                (j != k && dist(cur.id[k], i) > D)) {
                continue;
            }
        }
    }

    state tmp = cur;
    double movedist = dist(cur.id[j], i);
    tmp.dist += movedist;
    tmp.id[j] = i;
    sort(tmp.id, tmp.id + 3);

    // seen should be INF-initialised
    if (seen[tmp.id[0]][tmp.id[1]][tmp.id[2]] > tmp.dist) {
        pq.push(tmp);
        seen[tmp.id[0]][tmp.id[1]][tmp.id[2]] = tmp.dist;
    }
}
}
}
}

```

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Problem Statement** There are N cities, each in one of C countries. There are two modes of travel.
 - There are A one-way direct flight routes connecting two cities, u_i, v_i with weight w_i .
 - There are B one-way inter-country flights, connecting two countries, a_i, b_i with a weight w_i . These flights can be boarded from any city in the source country and disembarked from in any city in the destination country.
- Find the shortest distance from city 1 to city N .

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Input**

First line, 4 integers N, C, A, B . $1 \leq N, C, A, B \leq 100,000$.

Next line, N integers, c_i , denoting the country the i -th city is in.

Next A lines each with 3 integers

$u_i, v_i, w_i, 1 \leq u_i, v_i \leq N, 1 \leq w_i \leq 10^9$.

Next B lines each with 3 integers

$a_i, b_i, w_i, 1 \leq a_i, b_i \leq C, 1 \leq w_i \leq 10^9$.

- **Output** A single integer, the shortest distance from city $1 \rightarrow N$, or -1 if impossible.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Sample Input:**

4 3 2 2

1 2 2 3

1 4 100

2 4 20

1 2 50

1 3 80

- **Sample Output:** 70

- **Explanation:** Fly with an intercountry flight of cost 50 to city 2. Then take a direct flight with cost 20 to city 4.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- How to view this as a graph? Without inter-country flights, this is routine.
- With inter-country flights, we could just generate all edges between cities in country a_i and b_i .
- But this is too many edges.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Observation:** We should only consider inter-country flights originating from A the first time we reach a city in A .
- Similarly, we should only consider inter-country flights to B once.
- This sounds just like how we treat cities in Dijkstra's algorithm.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- To encode this, we should treat countries just like cities in our graph, they should have nodes. Be careful, we need 2 nodes per country, one to encode outgoing flights and one to encode incoming flights.
- The associated edges are natural.
 - Cities go to the “outgoing” node for their country.
 - The “incoming” node for a country goes to all cities in that country.
 - “Outgoing” country nodes connect by inter-country flights to “incoming” country nodes.
- $O(N + C)$ nodes, $O(N + A + B)$ edges. Dijkstra's algorithm now runs in time.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

• Implementation

```
#include <iostream>
#include <queue>
#include <utility>
#include <vector>
using namespace std;

typedef pair<int, long long> edge;

const int MAXN = 100100, MAXC = 100100;
int N, C, A, B;
vector<edge> edges[MAXN+2*MAXC]; // (dest, dist)

// city nodes are at 1..N
inline int city (int i) { return i; }
// "outgoing" country nodes are at N+1..N+C
inline int dep (int j) { return N + j; }
// "incoming" country nodes are at N+C+1..N+C+C
inline int arr (int j) { return N + C + j; }

// need bigger INF than usual, about 1e15
long long dist[MAXN+2*MAXC];
void dijkstra (int s);
```

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

• Implementation (continued)

```
int main(void) {
    cin >> N >> C >> A >> B;
    for (int i = 1; i <= N; i++) { // cities
        int j; cin >> j; // country
        // can go to your country's "intercountry departures" for free
        edges[city(i)].emplace_back(dep(j), 0);
        // can come from your country's "intercountry arrivals" for free
        edges[arr(j)].emplace_back(city(i), 0);
    }
    for (int i = 0; i < A; i++) { // intercity flights
        int a, b; long long w;
        cin >> a >> b >> w;
        edges[city(a)].emplace_back(city(b), w);
    }
    for (int i = 0; i < B; i++) { // intercountry flights
        int a, b; long long w;
        cin >> a >> b >> w;
        edges[dep(a)].emplace_back(arr(b), w);
    }
    dijkstra(city(1));
    cout << dist[city(n)] << '\n';
}
```

Example Problem: Escape From Enemy Territory51

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Problem Statement** You are at some position on a grid and wish to reach your safe house at some other location on the grid. However, also on certain cells on the grid are enemy safe houses, which you do not want to go near. What is the maximum possible Manhattan distance you can stay away from every enemy safe house, and still be able to reach your own safe house?
When there are multiple paths that keep the same distance from the enemy safe houses, print the shortest one.

Example Problem: Escape From Enemy Territory52

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Input** An $N \times M$ grid ($1 \leq N, M, \leq 1000$), and the location of your starting point, your safe house, and all the enemy safe houses. There are up to 10,000 enemy safe houses.
- **Output** Two integers, the maximum distance that you can stay away from every enemy safe house and still be able to reach your safe house, and the shortest length of such a path.

Example Problem: Escape From Enemy Territory53

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit Graphs

- If there was no restriction stating that you must stay as far away from the enemy safe houses as possible, this would be a simple shortest path problem on a grid.
- What if we already knew how far we need to stay away from each enemy safe house?

Example Problem: Escape From Enemy Territory⁵⁴

Shortest Paths

Single Source Shortest Paths

Dijkstra's Algorithm

Bellman-Ford Algorithm

All Pairs Shortest Paths

Implicit Graphs

- Call the distance that we know we need to stay away from the enemy safe houses X .
- We just need to BFS out from every enemy safe house to a distance of X squares, marking all of those squares as unusable. Just marking them as seen will suffice.
 - To avoid BFS from each of several enemy safe houses, instead do these searches simultaneously.
 - It might help to think of it as a BFS from a dummy node which has an edge to all enemy safe houses.
- Then we can find the answer with a simple BFS from the starting point. It will ignore the squares that are too close to enemy safe houses because we've marked them as seen.

Example Problem: Escape From Enemy Territory55

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- How do we view our original optimisation problem in terms of this decision problem?
- Our simpler problem is a decision problem because we answer whether or not it's possible to get from the starting point to the safe house with distance X .
- The original problem is an optimisation problem because it requires a 'best' answer.

Example Problem: Escape From Enemy Territory56

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Observe that if we can stay X distance away from the enemy safe houses, then any smaller distance is also feasible, and if we cannot stay X distance away, then any larger distance is also infeasible.
- This monotonicity allows us to binary search for the largest X such that we can still reach our safe house from our starting point, which we check using the BFS procedure outlined earlier.
- **Complexity** Each check takes $O(NM)$ time, and we need to perform $\log X_{MAX} = \log(N + M)$ of these checks in our binary search, so this algorithm takes $O(NM \log(N + M))$ total.

Example Problem: Escape From Enemy Territory57

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

● Implementation

```
// up down left right in a grid
const int di[4] = { -1, 1, 0, 0 };
const int dj[4] = { 0, 0, -1, 1 };

vector<pair<int,int>> enemies; // assume filled in
// search from all enemy safe houses to find
// each square's minimum distance to an enemy safe house
queue<pair<int, int>> q;
for (auto enemy : enemies) {
    q.push(enemy);
    dist_to_enemy[enemy.first][enemy.second] = 0;
}

while (!q.empty()) {
    pair<int, int> cur = q.front(); q.pop();
    int i = cur.first, j = cur.second;
    for (int d = 0; d < 4; d++) { // try all neighbours
        int ni = i + di[d], nj = j + dj[d];
        // if off board, ignore
        if (ni < 0 || ni >= N || nj < 0 || nj >= M) { continue; }
        // if seen, ignore
        if (dist_to_enemy[ni][nj] != INF) { continue; }
        dist_to_enemy[ni][nj] = dist_to_enemy[i][j] + 1;
        q.emplace(ni, nj);
    }
}
```

Example Problem: Escape From Enemy Territory58

Shortest Paths

Single Source Shortest Paths

Dijkstra's Algorithm

Bellman-Ford Algorithm

All Pairs Shortest Paths

Implicit Graphs

● Implementation (continued)

```
// binary search // answer in (lo, hi]
int lo = -1, hi = min(dist_to_enemy[i1][j1], dist_to_enemy[i2][j2]), sol =
    -1;
while (lo != hi) {
    int X = (lo + hi + 1) / 2;
    // BFS, since the edges are unit weight
    vector<vector<int>> dist(N, vector<int>(M, INF));
    dist[i1][j1] = 0;
    q.emplace(i1, j1);

    while (!q.empty()) {
        int i = q.front().first, j = q.front().second; q.pop();
        for (int d = 0; d < 4; ++d) {
            int ni = i + di[d], nj = j + dj[d];
            if (ni < 0 || ni >= N || nj < 0 || nj >= M) { continue; }
            if (dist_to_enemy[ni][nj] < X) { continue; }
            if (dist[ni][nj] != INF) { continue; }
            dist[ni][nj] = dist[i][j] + 1;
            q.emplace(ni, nj);
        }
    }

    if (dist[i2][j2] == INF) {
        hi = X - 1;
    } else {
        lo = X;
        sol = dist[i2][j2];
    }
}
```

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Given m inequalities of the form $x_i - x_j \leq c_k$, are there real numbers $x_1, x_2, \dots, x_n$ that satisfy those inequalities?
- What does this have to do with shortest paths??
- This can actually be solved in $O(nm)$ using Bellman-Ford!

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- We need to transform the problem into a graph problem, by creating a graph from the difference constraints.
- Create a vertex for every variable, and for every constraint $x_i - x_j \leq c_k$, create an edge from vertex j to vertex i with weight c_k .
- Then, create a source s that has a infinity-weighted edge to every other vertex.
 - There must be some outgoing edge from s so that the paths we're measuring actually exist.
 - An infinity-weight edge enforces that all x_i will be at most this infinity (as they are shortest paths, and we now have a path of weight INF).
 - Any other constant would have worked too; any constant offset applied to all x_i doesn't change anything.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- If we run Bellman-Ford from this source vertex and examine the shortest paths from the source, we obtain a solution.
- For every edge from j to i , from its definition we know that the length of the shortest path to i , say d_i , and the length of the shortest path to j , d_j , satisfy the inequality $d_i \leq d_j + c_k$.
- We can rewrite $d_i \leq d_j + c_k$ as $d_i - d_j \leq c_k$, which means that the distances from the source satisfy the difference constraints.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- What happens when there is a negative weight cycle?
- What happens when there is no solution to our system of difference constraints?

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- When we have a negative weight cycle, we have a set of edge inequalities with the variables in that cycle:

$$x_1 - x_2 \leq c_1$$

$$x_2 - x_3 \leq c_2$$

$$x_3 - x_1 \leq c_3$$

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- If we add all these inequalities together, because they form a cycle, all the variables cancel out, giving

$$0 \leq c_1 + c_2 + \dots + c_k,$$

where $c_1, c_2, \dots, c_k$ are the edge weights on our cycle.

- Since the cycle is negative, the RHS adds up to a negative number. Thus we have a contradiction, so there is no solution.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- **Problem Statement** Given a weighted directed graph with non-negative edge weights, what is the length of the second shortest path on the graph from s to t ? The second shortest path is the walk with the shortest length that is distinct from the shortest path. In particular, it is the same length as the shortest path if and only if there are multiple shortest paths.
- **Input** A weighted directed graph with n vertices and m edges ($1 \leq n, m \leq 100,000$).
- **Output** A single integer, the length of the second shortest walk.

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit Graphs

- Observe that the second shortest walk will differ from a given shortest path by at least one edge.
- Then after finding the shortest path, we can iterate over all edges on the shortest path, and see what happens if we go along a different edge instead.

Shortest Paths

Single Source Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs Shortest Paths

Implicit
Graphs

- Let the shortest path from u to v have length d_{uv} . If some edge (u, v) is blocked, and we take some other edge (u, w) instead, with weight e_{uw} we get a path of length $d_{su} + e_{uw} + d_{wt}$.
- Since we're iterating over all smallest possible differences to the shortest path, one of these distances will be our answer.
- So we just try them all (i.e. all choices of (u, w)) and take the one of minimum length.
- How do we efficiently compute all the d_{wt} ?

Shortest Paths

Single Source
Shortest Paths

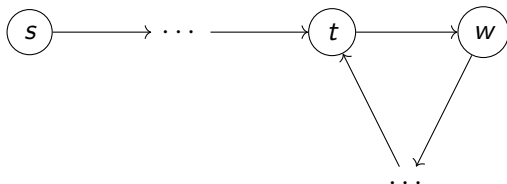
Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

- Edge case:



- Second shortest walk diverges *at the end of* the shortest walk.
- Make sure to try $u = t$, i.e. walks of the form $s \rightarrow \dots \rightarrow t \rightarrow w \rightarrow \dots \rightarrow t$.

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

● Implementation

```
typedef tuple<int, int, int> state; // dist, vertex, previous
typedef pair<int, int> edge; // dist, vertex
vector<edge> adj[N]; // dist, dest
vector<edge> adj_r[N]; // dist, origin

// dist1[u] = shortest distance from s to u
vector<int> dist1(n+1, INF), pred(n+1);
priority_queue<state, vector<state>, greater<state>> pq;
pq.emplace(0, s, 0);
while (!pq.empty()) {
    state cur = pq.top(); pq.pop();
    int d = get<0>(cur), v = get<1>(cur), u = get<2>(cur);
    // since C++17: auto [d, v, u] = cur;
    if (dist1[v] == INF) {
        dist1[v] = d;
        pred[v] = u;
        for (edge nxt : adj[v]) {
            int l = nxt.first, w = nxt.second; // v -> w of weight l
            // since C++17: auto [l, w] = nxt;
            if (dist1[w] != INF) { pq.emplace(d+l, w, v); }
        }
    }
}

// to be continued
```

Shortest Paths

Single Source
Shortest Paths

Dijkstra's
Algorithm

Bellman-Ford
Algorithm

All Pairs
Shortest Paths

Implicit
Graphs

Implementation (continued)

```
// dist2[u] = shortest distance from u to t
vector<int> dist2(n+1, INF);
priority_queue<state, vector<state>, greater<state>> pq_r;
pq_r.emplace(0, t, 0);
while (!pq_r.empty()) {
    state cur = pq_r.top(); pq_r.pop();
    int d = get<0>(cur), v = get<1>(cur); // don't need prev
    if (dist2[v] == INF) {
        dist2[v] = d;
        for (edge nxt : adj_r[v]) { // reverse graph
            int l = nxt.first, w = nxt.second; // v -> w of weight l
            if (dist2[w] != INF) { pq_r.emplace(d+l, w, v); }
        }
    }
}

int res = INF;
int u = t, v = 0; // special case for diverging at the end
while (u != 0) { // u -> v is an edge in the shortest walk
    for (edge e : adj[u]) { // try this edge instead of it
        int l = e.first, w = e.second;
        if (w != v) { res = min(res, dist1[u] + l + dist2[v]); }
    }
    v = u;
    u = pred[u];
}
```